PRAKTIKUM 4 PBO

Nama : Dzia Azizi Hamid

NIM : 251511010

Kelas : 2A-D3

Tujuan Pembelajaran

1. Menjelaskan hubungan superclass-subclass berdasarkan code yang benar-benar dijalankan.
2. Menerapkan inheritance dengan extends tanpa menggandakan state dan behavior yang sudah dimiliki superclass.
3. Menggunakan constructor subclass dan super(...) untuk menginisialisasi bagian object yang berasal dari superclass.
4. Membedakan inherited method dan overridden method melalui perubahan output program.
5. Menggunakan @Override sebagai pemeriksaan compiler terhadap niat overriding.
6. Menggunakan super.method(...) ketika subclass tetap membutuhkan behavior milik superclass.
7. Menjaga encapsulation pada hierarchy class: field private tetap diakses melalui behavior yang disediakan class pemiliknya.
8. Menerapkan inheritance, override, dan super pada kasus tugas sebelumnya.

Praktikum 1 – Circle dan Cylinder

Step 1 - Bangun Circle versi paling sederhana

Pada langkah pertama dibuat class Circle yang memiliki dua state yaitu radius bertipe double dan color bertipe String. Kedua atribut dibuat private untuk menjaga encapsulation.

Class Circle memiliki dua constructor. Constructor tanpa parameter memberikan nilai awal radius = 1.0 dan color = "red", sedangkan constructor Circle(double radius) memungkinkan nilai radius ditentukan pada saat object dibuat.

Selain itu, class Circle memiliki method getRadius() untuk mengambil nilai radius, getArea() untuk menghitung luas lingkaran, serta toString() untuk menampilkan informasi object dalam bentuk teks.

Hasil pengujian:

Object c1 dibuat menggunakan constructor tanpa parameter sehingga memiliki radius 1.0. Object c2 dibuat menggunakan constructor dengan parameter 2.5 sehingga memiliki radius 2.5. Kedua object memiliki warna awal "red".

```java
public class Circle {
    private double radius;
    private String color;

    public Circle() {
        radius = 1.0;
        color = "red";
    }

    public Circle(double radius) {
        this.radius = radius;
        color = "red";
    }

    public double getRadius() {
        return radius;
    }

    public double getArea() {
        return radius * radius * Math.PI;
    }

    @Override
    public String toString() {
        return "Circle[radius=" + radius + ", color=" + color + "]";
    }
}
```

```
=== Circle c1 ===
Radius : 1.0
Area   : 3.141592653589793
Object : Circle[radius=1.0, color=red]

=== Circle c2 ===
Radius : 2.5
Area   : 19.634954084936208
Object : Circle[radius=2.5, color=red]
```

Checkpoint 1:

1. State pada circle adalah:

   Radius

   Color

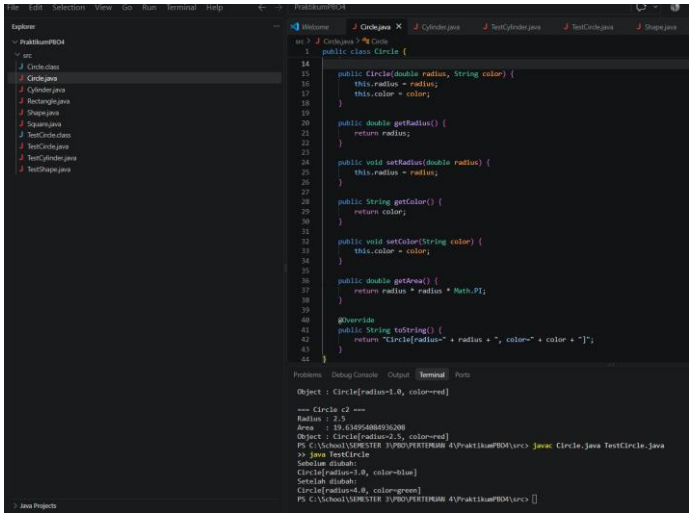
2. Behavior pada circle adalah:
   Circle()
   Circle(double radius)
   getRadius()
   getArea()
   toString()

Perbedaan state antara object c1 dan c2 terdapat pada nilai radius. Object c1 memiliki radius 1.0, sedangkan c2 memiliki radius 2.5. Keduanya memiliki warna yang sama, yaitu "red".

Step 2 – Lengkapi encapsulation Circle

Pada langkah ini class Circle dilengkapi dengan constructor yang menerima parameter radius dan color, serta method setter dan getter. Atribut radius dan color tetap dideklarasikan sebagai private, sehingga perubahan nilai dari luar class dilakukan melalui method public.

Penggunaan setter menunjukkan bahwa state object tetap terlindungi oleh prinsip encapsulation. Kode di luar class tidak dapat mengubah field private secara langsung, tetapi dapat mengakses atau mengubahnya melalui method yang disediakan class.



Pengujian encapsulation pada class Circle. Nilai radius dan color berhasil diubah melalui method setter tanpa mengakses field private secara langsung.

Checkpoint 2: Kode di luar Circle tidak dapat mengubah radius secara langsung karena radius memiliki access modifier private. Method public seperti setRadius() menjadi penghubung yang memungkinkan object mengubah state-nya tanpa membuka field tersebut secara langsung.

Step 3 – Prediksi sebelum membuat Cylinder

1. Apakah radius dan color perlu ditulis ulang sebagai file Cylinder?
   Tidak perlu. Karena Cylinder akan menjadi subclass dari circle, maka state yang sudah memiliki oleh circle tidak perlu diduplikasi di cylinder.
2. Behavior apa dari Circle yang masih berguna untuk Cylinder?
   - getRadius()
   - setRadius()
   - getColor()
   - getArea()
   - toString()
3. State apa yang hanya dimiliki Cylinder?
   Height

Step 4 – Buat Cylinder sebagai subclass Circle

Pada langkah ini dibuat class Cylinder yang merupakan subclass dari Circle menggunakan keyword extends. Class Cylinder menambahkan state baru berupa height, sedangkan radius dan color tidak ditulis ulang karena sudah dimiliki oleh superclass Circle. Constructor Cylinder menggunakan super() dan super(radius) untuk memanggil constructor pada superclass. Dengan demikian, bagian object yang berasal dari Circle dapat diinisialisasi terlebih dahulu sebelum bagian khusus Cylinder.

```java
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        return getArea() * height;
    }
}
```

Step 5 – Uji inherited method

Pada langkah ini dibuat tiga object Cylinder dengan constructor yang berbeda. Hasil pengujian menunjukkan bahwa object Cylinder dapat memanggil method getRadius() dan getArea() walaupun kedua method tersebut tidak ditulis di dalam class Cylinder. Hal ini terjadi karena Cylinder mewarisi behavior yang dapat diakses dari superclass Circle.

```
c1
radius = 1.0
height = 1.0
base area = 3.141592653589793
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 3.141592653589793
volume = 31.41592653589793

c3
```

Checkpoint 3: c3.getRadius() dan c3.getArea() dapat dipanggil karena Cylinder merupakan subclass dari Circle. Method getRadius() dan getArea() diwariskan dari superclass sehingga dapat digunakan oleh object Cylinder tanpa perlu menyalin ulang kode method tersebut.

Step 6 – Amati constructor chain dan fungsi super

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Circle.java Cylinder.java TestCylinder.java
>> java TestCylinder
Circle(double) dipanggil
Cylinder(double, double) dipanggil
radius = 2.0
height = 10.0
```

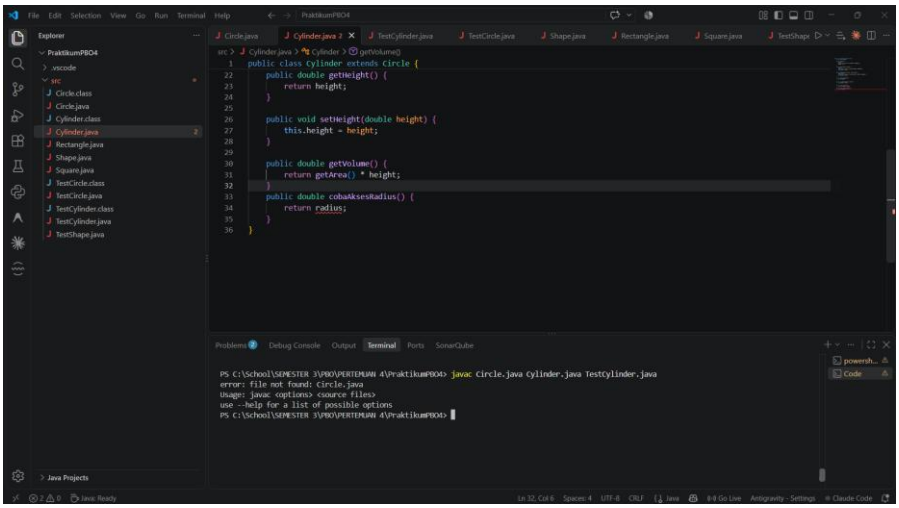
Pada langkah ini ditambahkan output sementara pada constructor Circle dan Cylinder untuk mengamati urutan pemanggilan constructor. Ketika object new Cylinder(2.0, 10.0) dibuat, output menunjukkan bahwa constructor Circle(double) dijalankan terlebih dahulu, kemudian constructor Cylinder(double,double). Hal ini membuktikan bahwa bagian superclass diinisialisasi sebelum bagian subclass.

Ketika super(radius) dihapus, Java secara otomatis memanggil constructor super(). Karena class Circle memiliki constructor tanpa parameter, program tetap dapat dijalankan. Namun nilai radius berubah menjadi 1.0 karena constructor default Circle() yang digunakan, bukan Circle(double radius). Setelah observasi selesai, super(radius) dikembalikan agar nilai radius 2.0 diteruskan dengan benar ke superclass.

Checkpoint 4

Bagian Circle dikerjakan lebih dahulu daripada bagian Cylinder. Hal tersebut dibuktikan dari output Circle(double) dipanggil yang muncul sebelum Cylinder(double,double) dipanggil. Pemanggilan super(radius) menyebabkan constructor superclass dijalankan terlebih dahulu sebelum proses inisialisasi pada subclass dilanjutkan.

Step 7 - Uji batas encapsulation pada subclass



Pada langkah ini dilakukan percobaan mengakses field radius secara langsung dari class Cylinder. Walaupun Cylinder merupakan subclass dari Circle, compiler tetap menolak akses tersebut karena radius memiliki access modifier private. Hal ini menunjukkan bahwa inheritance tidak menghilangkan aturan encapsulation. Untuk memperoleh nilai radius, subclass harus menggunakan behavior yang disediakan superclass, yaitu getRadius().

Checkpoint 5

Keyword extends memungkinkan subclass menggunakan behavior yang diwariskan dari superclass, tetapi field yang dideklarasikan private tetap hanya dapat diakses langsung dari dalam class pemiliknya. Subclass harus menggunakan getter, setter, atau method public/protected lain yang disediakan superclass.

Step 8 - Override getArea(), lalu temukan bug pada getVolume()

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Circle.java Cylinder.java TestCylinder.java
>> java TestCylinder
Circle(double) dipanggil
Cylinder(double, double) dipanggil
radius = 2.0
height = 10.0
area = 150.79644737231007
volume = 1507.9644737231006
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

```java
public class Cylinder extends Circle {
    public Cylinder(double radius, double height) {
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        return getArea() * height;
    }
    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height
                + 2 * Math.PI * getRadius() * getRadius();
    }
}
```

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Circle.java Cylinder.java TestCylinder.java
>> java TestCylinder
Circle(double) dipanggil
Cylinder(double, double) dipanggil
radius = 2.0
height = 10.0
area = 150.79644737231007
volume = 125.66370614359172
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

Pada langkah ini method getArea() pada class Cylinder dioverride sehingga menghitung luas permukaan silinder. Setelah perubahan tersebut, method getVolume() menghasilkan nilai yang salah karena sebelumnya menggunakan pemanggilan getArea(). Pemanggilan tersebut sekarang mengarah ke implementasi getArea() milik Cylinder, bukan lagi implementasi milik Circle.

Untuk memperbaikinya, method getVolume() diubah menjadi menggunakan super.getArea(). Dengan demikian, perhitungan volume secara eksplisit menggunakan luas lingkaran dari superclass Circle sebagai luas alas silinder.

Checkpoint 6

Setelah overriding, getArea() pada object Cylinder menjalankan implementasi milik Cylinder. Jika subclass masih membutuhkan implementasi superclass, maka dapat digunakan super.getArea(). Dalam kasus ini super.getArea() diperlukan agar getVolume() tetap menggunakan luas alas lingkaran dan bukan luas permukaan silinder.

Step 9 – Override toString() dan uji @override

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Circle.java Cylinder.java TestCylinder.java
>> java TestCylinder
Circle(double) dipanggil
Cylinder(double, double) dipanggil
Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Circle.java Cylinder.java TestCylinder.java
Cylinder.java:40: error: toStrng() in Cylinder does not override or implement a method from a supertype
    @Override
    ^
1 error
```

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Circle.java Cylinder.java TestCylinder.java
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

Pada langkah ini method toString() pada class Cylinder dioverride. Method tersebut menggunakan super.toString() agar representasi object tetap memanfaatkan informasi yang berasal dari superclass Circle, kemudian menambahkan informasi height dari Cylinder.

Selanjutnya nama method sengaja diubah menjadi toStrng() dengan tetap mempertahankan anotasi @Override. Compiler menghasilkan error karena tidak terdapat method pada superclass yang sesuai untuk dioverride. Ketika anotasi @Override dihapus, method tersebut dapat dikompilasi karena dianggap sebagai method baru. Percobaan ini menunjukkan bahwa @Override membantu compiler memeriksa apakah method benar-benar meng-override method dari superclass.

Checkpoint 7

@Override tidak menyebabkan inheritance terjadi. Anotasi tersebut digunakan untuk memberi tahu compiler bahwa sebuah method dimaksudkan untuk meng-override method dari superclass. Jika nama method, parameter, atau signature tidak sesuai, compiler dapat mendeteksi kesalahan.

Step 10 – Buat superclass Shape



```java
public class Shape {
    private String color;
    private boolean filled;

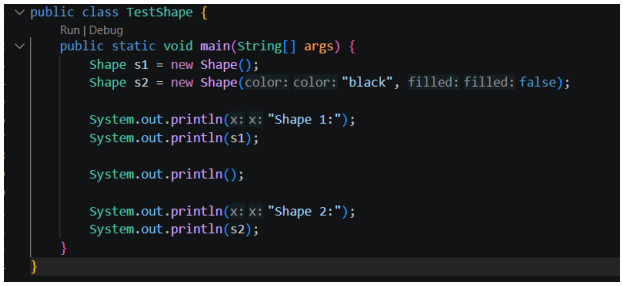
    public Shape() {
        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

    public boolean isFilled() {
        return filled;
    }
```



```java
public class TestShape {
    Run | Debug
    public static void main(String[] args) {
        Shape s1 = new Shape();
        Shape s2 = new Shape(color: color: "black", filled: filled: false);

        System.out.println(x: x: "Shape 1:");
        System.out.println(s1);

        System.out.println();

        System.out.println(x: x: "Shape 2:");
        System.out.println(s2);
    }
}
```

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Shape.java TestShape.java
>> java TestShape
Shape 1:
Shape[color=green, filled=true]

Shape 2:
Shape[color=black, filled=false]
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

Pada langkah ini dibuat class Shape sebagai superclass yang menyimpan state umum berupa color dan filled. Class ini memiliki constructor default dan constructor berparameter, serta getter dan setter untuk mengakses state tersebut. Method toString() dioverride untuk menampilkan informasi object Shape.

Checkpoint 8

State yang cukup umum untuk dimiliki oleh Circle maupun Rectangle adalah color dan filled, karena kedua bentuk tersebut dapat memiliki warna dan kondisi terisi atau tidak terisi.

Step 11 - Buat Circle sebagai specialization dari Shape

```
=== Circle c1 ===
Circle[radius=1.0, Shape[color=green, filled=true]]
Area      = 3.141592653589793
Perimeter = 6.283185307179586

=== Circle c2 ===
Circle[radius=2.0, Shape[color=green, filled=true]]
Area      = 12.566370614359172
Perimeter = 12.566370614359172

=== Circle c3 ===
Circle[radius=3.0, Shape[color=blue, filled=true]]
Color     = blue
Filled    = true
Area      = 28.274333882308138
Perimeter = 18.84955592153876
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

```java
public class Circle extends Shape {
    private double radius;

    public Circle() {
        super();
        radius = 1.0;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }

    public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
```

Pada langkah ini class Circle diubah menjadi subclass dari Shape menggunakan extends Shape. State yang bersifat umum, yaitu color dan filled, diperoleh dari superclass Shape, sedangkan state khusus Circle tetap berupa radius. Constructor Circle menggunakan super() atau super(color, filled) untuk menginisialisasi bagian object yang berasal dari superclass. Circle juga memiliki behavior khusus berupa getArea() dan getPerimeter().

Checkpoint 9

Behavior yang berasal dari shape: getcolor(), setcolor(), isFilled(), setFilled(). toString dari shape juga dipanggil melalui super.toString(). Behavior milik Circle: getRadius(), setRadius(), getArea(), getPerimeter(). Sedangkan toString() pada circle Adalah override, karena circle menyediakan implementasinya sendiri.

Step 12 - Buat Rectangle sebagai specialization dari Shape

```
=== Rectangle r1 ===
Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]
Area      = 1.0
Perimeter = 4.0

=== Rectangle r2 ===
Rectangle[width=3.0, length=4.0, Shape[color=green, filled=true]]
Area      = 12.0
Perimeter = 14.0

=== Rectangle r3 ===
Rectangle[width=5.0, length=6.0, Shape[color=yellow, filled=true]]
Color     = yellow
Filled    = true
Area      = 30.0
Perimeter = 22.0
```

Pada langkah ini dibuat class Rectangle sebagai subclass dari Shape. State umum berupa color dan filled diperoleh dari superclass, sedangkan Rectangle memiliki state khusus yaitu width dan length. Constructor Rectangle menggunakan super() atau super(color, filled) untuk menginisialisasi bagian superclass. Class Rectangle juga memiliki method getArea() untuk menghitung luas dan getPerimeter() untuk menghitung keliling.

Checkpoint 10

Kemiripan antara Circle dan Rectangle yang ditempatkan pada Shape adalah state color dan filled beserta behavior getter dan setter-nya. Sementara itu, state dan behavior yang berhubungan dengan bentuk masing-masing tetap berbeda. Circle memiliki radius, sedangkan

Rectangle memiliki width dan length. Perhitungan area dan perimeter masing-masing class juga berbeda sesuai bentuknya.

Step 13 - Buat Square dari Rectangle

```
=== Square s1 ===
Square[side=1.0, Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]]
Side      = 1.0
Area      = 1.0
Perimeter = 4.0

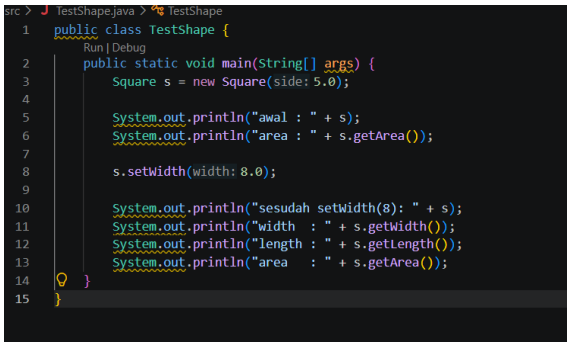
=== Square s2 ===
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
Side      = 5.0
Area      = 25.0
Perimeter = 20.0

=== Square s3 ===
Square[side=4.0, Rectangle[width=4.0, length=4.0, Shape[color=blue, filled=true]]]
Color     = blue
Filled    = true
```

Pada langkah ini dibuat class Square sebagai subclass dari Rectangle. Class Square tidak menambahkan field baru karena menggunakan width dan length yang sudah dimiliki oleh Rectangle. Constructor Square mengirim nilai sisi yang sama untuk width dan length melalui pemanggilan super(...). Method getSide() menggunakan nilai width, sedangkan setSide() mengubah width dan length secara bersamaan agar keduanya tetap sama.

Step 14 - Temukan pelanggaran aturan Square

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Shape.java Rectangle.java Square.java TestShape.java
>> java TestShape
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=5.0, Shape[color=green, filled=true]]]
width  : 8.0
length : 5.0
area   : 40.0
```



Pada langkah ini dibuat object Square dengan sisi awal 5.0. Kemudian method setWidth(8.0) yang diwarisi dari Rectangle dipanggil. Hasil pengujian menunjukkan bahwa nilai width berubah menjadi 8.0, sedangkan length tetap 5.0. Kondisi ini menyebabkan object tidak lagi merepresentasikan persegi karena kedua sisi tidak sama. Percobaan ini menunjukkan bahwa

behavior yang diwarisi secara teknis dapat digunakan, tetapi belum tentu sesuai dengan aturan subclass yang lebih spesifik.

Checkpoint 11

Inheritance dapat memberikan method yang tersedia dari superclass, tetapi subclass dapat memiliki aturan tambahan yang harus dijaga. Pada Square, method setWidth() dari Rectangle dapat merusak kondisi width == length, sehingga diperlukan overriding agar aturan object Square tetap valid.

Step 15 - Override setter agar Square tetap valid

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src> javac Shape.java Rectangle.java Square.java TestShape.java
>> java TestShape
=== Kondisi awal ===
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
width     = 5.0
length    = 5.0
area      = 25.0
perimeter = 20.0

=== Setelah setWidth(8.0) ===
Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
width     = 8.0
length    = 8.0
area      = 64.0
perimeter = 32.0

=== Setelah setLength(3.0) ===
Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
width     = 3.0
length    = 3.0
area      = 9.0
perimeter = 12.0
```

```java
@Override
public void setWidth(double side) {
    super.setWidth(side);
    super.setLength(side);
}

@Override
public void setLength(double side) {
    super.setWidth(side);
    super.setLength(side);
}
```

Pada langkah ini method setWidth() dan setLength() pada class Square dioverride. Setiap perubahan pada salah satu sisi akan mengubah width dan length secara bersamaan. Dengan demikian, aturan bahwa sisi-sisi Square harus memiliki panjang yang sama tetap terjaga.

Pengujian menunjukkan bahwa setelah setWidth(8.0) dipanggil, nilai width dan length sama-sama menjadi 8.0. Setelah setLength(3.0) dipanggil, keduanya berubah menjadi 3.0.

Checkpoint 12

setWidth() dan setLength() perlu dioverride karena implementasi milik Rectangle hanya mengubah satu dimensi, sehingga dapat menyebabkan width dan length pada Square berbeda.

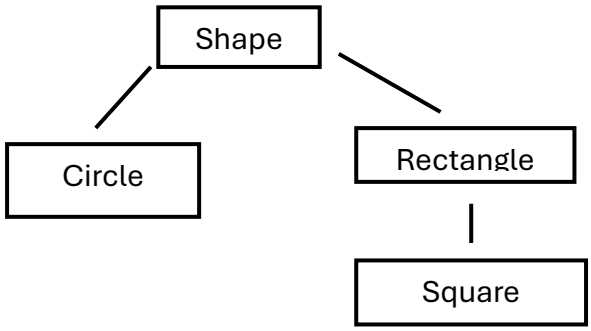
Sementara itu, getArea() dan getPerimeter() tidak perlu dioverride karena implementasi dari Rectangle tetap menghasilkan perhitungan yang benar selama width dan length dijaga selalu sama.

Step 16 - Test hierarchy secara utuh

```
src > TestShape.java > TestShape
1   public class TestShape {
        Run | Debug
2       public static void main(String[] args) {
3           Shape a = new Shape(color: "black", filled: false);
4           Circle b = new Circle(radius: 2.0, color: "blue", filled: true);
5           Rectangle c = new Rectangle(width: 3.0, length: 4.0, color: "yellow", filled: true);
6           Square d = new Square(side: 5.0, color: "green", filled: true);
7
8           System.out.println(a);
9           System.out.println(b + " area=" + b.getArea());
10          System.out.println(c + " area=" + c.getArea());
11          System.out.println(d + " area=" + d.getArea());
12      }
13  }
```

```
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 4\PraktikumPBO4\src>
```

Pada langkah ini seluruh hierarchy class diuji secara bersamaan. Object Shape, Circle, Rectangle, dan Square dibuat dengan constructor berparameter. Hasil pengujian menunjukkan bahwa setiap subclass dapat menggunakan state umum dari superclass sekaligus mempertahankan state dan behavior khusus masing-masing. Circle menghitung luas berdasarkan radius, Rectangle berdasarkan width dan length, sedangkan Square menggunakan implementasi Rectangle dengan aturan width dan length yang tetap sama.



Step 17 — Cari kandidat generalization.

Superclass yang dipilih: Barang

Subclass:

- BarangElektronik
- BarangNonElektronik

State/behavior yang sama dan cocok ditempatkan pada Barang:

kodeBarang

namaBarang

getKodeBarang()

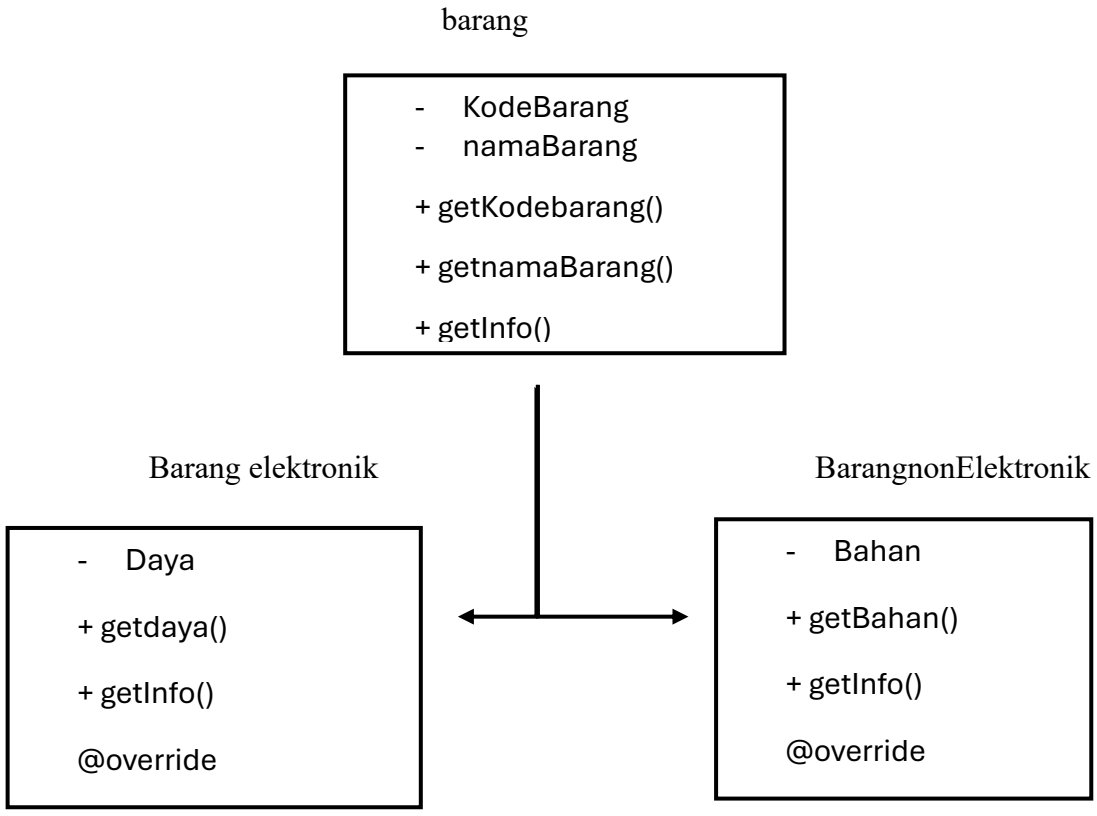
getNamaBarang()

Class	**State/behavior khusus**
BarangElektronik	daya, informasi barang elektronik
BarangNonElektronik	bahan, informasi barang non-elektronik

Pada kasus pencatatan inventaris, class Barang dipilih sebagai superclass karena dapat mewakili karakteristik umum berbagai jenis barang. Dua specialization yang dirancang adalah BarangElektronik dan BarangNonElektronik. Kedua subclass memiliki state umum berupa kodeBarang dan namaBarang, tetapi memiliki karakteristik khusus yang berbeda. BarangElektronik memiliki informasi khusus berupa daya, sedangkan BarangNonElektronik memiliki informasi berupa bahan.

Hubungan inheritance dinilai sesuai karena kalimat "BarangElektronik adalah sebuah Barang" dan "BarangNonElektronik adalah sebuah Barang" memiliki makna yang benar pada model inventaris. Sementara itu, class Lokasi tidak dimasukkan ke dalam hierarchy tersebut karena Lokasi bukan merupakan jenis dari Barang.

Step 18 - Gambar sebelum coding



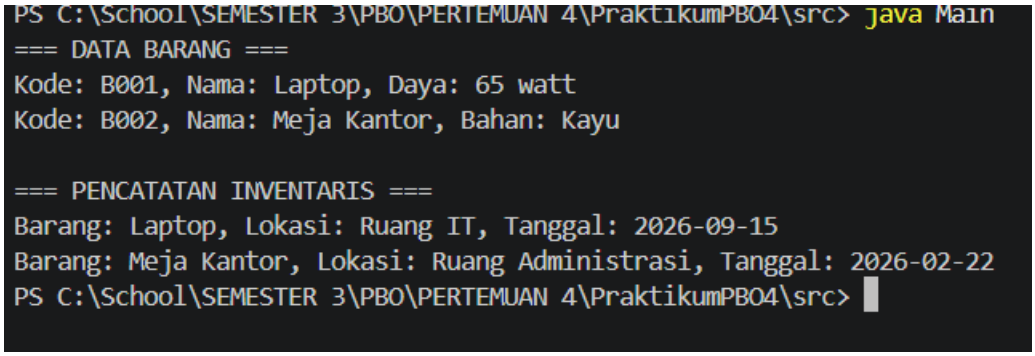
Jadi pembagiannya adalah:

- Superclass: Barang
- Subclass: BarangElektronik dan BarangNonElektronik
- State umum: kodeBarang, namaBarang
- Behavior umum: getKodeBarang(), getNamaBarang(), getInfo()
- State khusus BarangElektronik: daya
- State khusus BarangNonElektronik: bahan
- Method yang akan dioverride: getInfo()

Sebelum melakukan perubahan pada source code, dibuat hierarchy class untuk menentukan bagian yang bersifat umum dan bagian yang bersifat khusus. Class Barang digunakan sebagai superclass yang menyimpan state umum berupa kodeBarang dan namaBarang. Class BarangElektronik dan BarangNonElektronik menjadi subclass dari Barang.

BarangElektronik memiliki state khusus berupa daya, sedangkan BarangNonElektronik memiliki state khusus berupa bahan. Method getInfo() direncanakan untuk dioverride pada masing-masing subclass agar informasi yang ditampilkan dapat disesuaikan dengan jenis barang. Implementasi subclass juga direncanakan tetap memanfaatkan super.getInfo() agar behavior umum dari superclass tetap digunakan.

Step 19 - Implementasikan perubahan



Pada tahap implementasi, class Barang digunakan sebagai superclass. Dua subclass yang dibuat adalah BarangElektronik dan BarangNonElektronik. Kedua subclass menggunakan keyword extends Barang dan memanggil constructor superclass melalui super(...).

Class Barang memiliki method umum getInfo(). Method tersebut dioverride pada BarangElektronik dan BarangNonElektronik menggunakan anotasi @Override. Implementasi subclass tetap menggunakan super.getInfo() untuk mengambil informasi dasar dari superclass, kemudian menambahkan informasi khusus masing-masing subclass.

BarangElektronik menambahkan state daya, sedangkan BarangNonElektronik menambahkan state bahan. Semua field tetap menggunakan access modifier private sehingga prinsip encapsulation tetap dipertahankan.

Class PencatatanInventaris tetap menyimpan object menggunakan tipe Barang. Object BarangElektronik maupun BarangNonElektronik tetap dapat digunakan karena keduanya merupakan subclass dari Barang. Dengan demikian, fungsi utama dari tugas sebelumnya tetap dapat berjalan setelah dilakukan refactoring.



```java
public class Main {
    Run | Debug
    public static void main(String[] args) {

        BarangElektronik barang1 =
                new BarangElektronik(kodeBarang: "B001", namaBarang: "Laptop", daya: 65);

        BarangNonElektronik barang2 =
                new BarangNonElektronik(kodeBarang: "B002", namaBarang: "Meja Kantor", bahan: "Kayu");

        Lokasi lokasi1 =
                new Lokasi(kodeLokasi: "L001", namaLokasi: "Ruang IT");

        Lokasi lokasi2 =
                new Lokasi(kodeLokasi: "L002", namaLokasi: "Ruang Administrasi");

        PencatatanInventaris pencatatan1 =
                new PencatatanInventaris(
                        barang1,
                        lokasi1,
                        tanggalPencatatan: "2026-09-15");

        PencatatanInventaris pencatatan2 =
                new PencatatanInventaris(
                        barang2,
                        lokasi2,
```

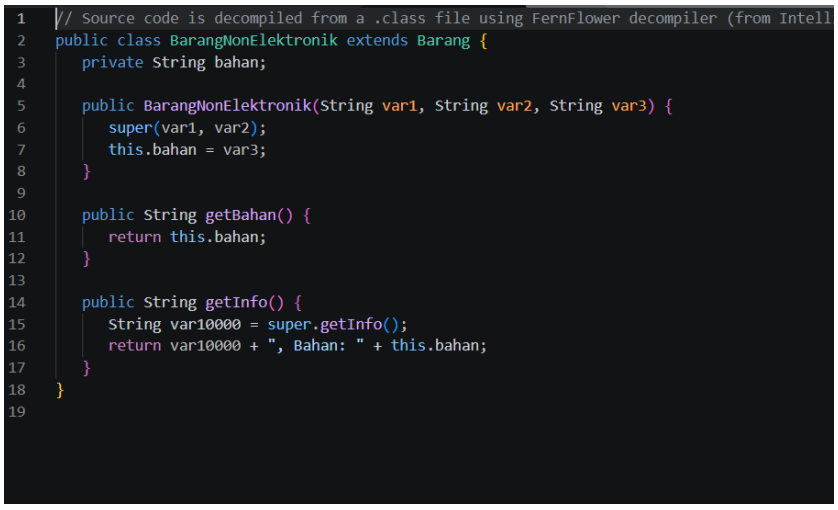


```java
public class BarangElektronik extends Barang {
    private int daya;

    public BarangElektronik(String kodeBarang, String namaBarang, int daya) {
        super(kodeBarang, namaBarang);
        this.daya = daya;
    }

    public int getDaya() {
        return daya;
    }

    @Override
    public String getInfo() {
        return super.getInfo() + ", Daya: " + daya + " watt";
    }
}
```



```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intell
public class BarangNonElektronik extends Barang {
   private String bahan;

   public BarangNonElektronik(String var1, String var2, String var3) {
      super(var1, var2);
      this.bahan = var3;
   }

   public String getBahan() {
      return this.bahan;
   }

   public String getInfo() {
      String var10000 = super.getInfo();
      return var10000 + ", Bahan: " + this.bahan;
   }
}
```

Step 20 - Uji sebelum menyatakan selesai

```
STEP 20 - PENGUJIAN KASUS INVENTARIS
====================================

=== T4-01 CONSTRUCTOR SUBCLASS ===
Barang elektronik berhasil dibuat:
Kode: B001, Nama: Laptop, Daya: 65 watt
Barang non-elektronik berhasil dibuat:
Kode: B002, Nama: Meja Kantor, Bahan: Kayu

=== T4-02 INHERITED METHOD ===
Nama barang1 dari getNamaBarang(): Laptop
Kode barang2 dari getKodeBarang(): B002

=== T4-03 OVERRIDE getInfo() ===
Kode: B001, Nama: Laptop, Daya: 65 watt
Kode: B002, Nama: Meja Kantor, Bahan: Kayu

=== T4-04 super.getInfo() ===
Elektronik : Kode: B001, Nama: Laptop, Daya: 65 watt
Non Elektronik : Kode: B002, Nama: Meja Kantor, Bahan: Kayu

=== T4-05 ENCAPSULATION ===
Kode melalui getter  : B001
Nama melalui getter  : Laptop
Daya melalui getter  : 65
Bahan melalui getter : Kayu

=== T4-06 DUA SUBCLASS ===
BarangElektronik:
Kode: B001, Nama: Laptop, Daya: 65 watt
BarangNonElektronik:
Kode: B002, Nama: Meja Kantor, Bahan: Kayu

=== T4-07 REGRESSION TUGAS LAMA ===
Barang: Laptop, Lokasi: Ruang IT, Tanggal: 2026-09-15
Barang: Meja Kantor, Lokasi: Ruang Administrasi, Tanggal: 2026-02-22
```

Self Assessment

Pertanyaan	Skor 1-5
Saya dapat menjelaskan extends dari code yang saya jalankan.	**3**
Saya dapat membedakan inherited method dan overridden method.	**3**
Saya memahami mengapa constructor subclass memanggil super(...).	**3**
Saya dapat menggunakan super.method(...) dengan alasan yang tepat.	**3**
Saya dapat menjaga encapsulation pada hierarchy class.	**3**
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	**3**

Refleksi

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?
   Circle layak menjadi class yang lebih umum karena Cylinder memiliki karakteristik dasar yang berasal dari lingkaran, seperti radius, color, dan perhitungan luas lingkaran. Cylinder kemudian menambahkan karakteristik khusus berupa height dan volume. Oleh karena itu, Circle dapat dijadikan superclass dan Cylinder sebagai subclass tanpa perlu menduplikasi state dan behavior yang sudah ada.
2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?
   Secara praktis, extends memungkinkan object Cylinder menggunakan method yang berasal dari Circle, seperti getRadius() dan getArea(), walaupun method tersebut tidak

ditulis ulang di dalam Cylinder. Hal ini menunjukkan bahwa subclass dapat menggunakan behavior yang diwarisi dari superclass.

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?
   Constructor Circle dijalankan karena object Cylinder juga memiliki bagian yang berasal dari superclass Circle. Saat object Cylinder dibuat, constructor superclass dijalankan terlebih dahulu melalui super() atau super(radius), kemudian constructor Cylinder melanjutkan proses inisialisasi bagian khususnya, yaitu height.
4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?
   Setelah getArea() dioverride pada Cylinder, method tersebut tidak lagi menghitung luas alas lingkaran, tetapi menghitung luas permukaan silinder. Sementara itu, getVolume() sebelumnya menggunakan: return getArea() * height;
   Akibatnya, volume dihitung menggunakan luas permukaan, bukan luas alas. Oleh karena itu, hasil volume menjadi salah.
5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?
   Ketika menulis super.getArea() atau super.toString(), saya secara eksplisit meminta program menggunakan implementasi method milik superclass, bukan implementasi yang sudah dioverride oleh subclass.
   Pada kasus Cylinder, super.getArea() digunakan agar perhitungan volume tetap menggunakan luas lingkaran dari Circle. Sedangkan super.toString() digunakan untuk mengambil informasi object dari superclass kemudian menambahkan informasi khusus subclass.
6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?
   Square perlu meng-override setWidth() dan setLength() karena sebuah persegi harus memiliki width dan length yang sama. Jika setter dari Rectangle digunakan langsung, salah satu nilai dapat berubah tanpa mengubah nilai lainnya sehingga object tidak lagi merepresentasikan persegi. Dengan overriding, perubahan pada salah satu sisi akan mengubah kedua sisi secara bersamaan.
7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.
   Tidak. Dua class yang mirip belum tentu memiliki hubungan inheritance. Inheritance sebaiknya digunakan jika terdapat hubungan “is-a” yang masuk akal. Pada kasus inventaris, BarangElektronik dapat dikatakan “adalah sebuah Barang”, sehingga cocok menjadi subclass dari Barang. Begitu juga BarangNonElektronik. Sebaliknya, Lokasi tidak cocok menjadi subclass dari Barang karena kalimat “Lokasi adalah sebuah Barang” tidak masuk akal. Prinsip ini juga sesuai dengan langkah generalization pada modul.
8. Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."
   Inheritance bukan sekadar mengurangi duplikasi, tetapi juga membentuk hubungan antara class yang umum dan class yang lebih spesifik, sehingga subclass dapat menggunakan kembali behavior dari superclass sekaligus menambahkan atau mengubah behavior sesuai kebutuhan.